TECHNICAL DOCUMENTATION

Person Name Finder

AI-Powered Executive Search System

Version	1.0.0
Date	March 2026
Stack	FastAPI · SearXNG · Groq LLM · Docker
Status	Production Ready

ABOUT THIS DOCUMENT

This document provides complete technical documentation for the Person Name Finder application — an AI-powered tool that identifies the current holder of any executive or managerial role at any company using web search and large language models.

1. Project Overview

Person Name Finder is a full-stack web application that automatically identifies the current holder of any job title or designation at any company. It combines self-hosted web search (SearXNG), multiple search engine fallbacks, and Groq-hosted large language models to deliver confident, verified answers in seconds.

Core Use Case

Enter a company name and a job designation (e.g., "Google" + "CEO") and the system returns the current person's full name, the source URL, and a confidence score — without any manual searching.

1.1 Key Features

- Single search: query one company + designation at a time with instant results
- Bulk search: upload a CSV of company/designation pairs and download a completed CSV
- AI-powered extraction: Groq LLM reads search snippets and extracts the person's name
- Confidence scoring: every result is scored 0–100 based on source trust and match quality
- Multi-pass search strategy: strict pass first, then relaxed fallback to maximise recall
- Model rotation: automatically cycles through multiple Groq models on rate-limit errors
- Mock mode: test the full UI without real API keys
- Docker deployment: one command to run the complete stack

1.2 Tech Stack

Layer	Technology	Purpose
Backend	FastAPI (Python 3.11)	REST API server
Search	SearXNG (Docker)	Self-hosted meta search engine
AI / LLM	Groq API (llama-3.1-8b)	Name extraction from snippets
Frontend	Vanilla HTML/CSS/JS	Browser-based UI
Rate Limiting	SlowAPI	Prevents API abuse
Data	Pandas + CSV	Bulk input/output handling
Deployment	Docker Compose	Container orchestration

2. Architecture

The application follows a clean layered architecture. The FastAPI backend orchestrates a pipeline of specialised modules: query generation → search → filtering → LLM extraction → verification → confidence scoring.

2.1 Project Structure

```
Person Name Finder searXNG/
├── docker-compose.yml          # Orchestrates API + SearXNG containers
├── Dockerfile                  # Python 3.11-slim image for the API
├── settings.yml                # SearXNG configuration
├── run_server.bat              # Windows dev startup script
└── app/
    ├── main.py                 # FastAPI app entry point, CORS, rate limiting
    ├── requirements.txt        # Python dependencies
    ├── .env                    # Environment variables (API keys, URLs)
    ├── index.html              # Frontend UI
    ├── app.js                  # Frontend JavaScript logic
    ├── style.css               # Frontend styles
    ├── routes/
    │   ├── search.py           # POST /search — single search endpoint
    │   └── bulk_search.py      # POST /bulk-search — CSV batch endpoint
    ├── services/
    │   ├── query_generator.py  # Generates multiple search query variants
    │   ├── searxng_client.py   # Async SearXNG HTTP client
    │   ├── duckduckgo_client.py# DuckDuckGo fallback client
    │   └── llm_processor.py    # Groq LLM integration & model rotation
    ├── agents/
    │   ├── result_filter_agent.py   # Strict & relaxed result filtering
    │   └── verification_agent.py    # Validates LLM output quality
    └── utils/
        ├── confidence_score.py      # Scores results 0-100
        └── csv_handler.py           # CSV parse & serialize helpers
```

2.2 Request Lifecycle

When a user submits a search, the following pipeline executes:

Step	Module	Action
1	routes/search.py	Receive POST /search, validate input, check MOCK_MODE and known corrections
2	query_generator.py	Generate 10 search query variants (quoted, LinkedIn, profile, etc.)
3	searxng_client.py	Query SearXNG with random engine subset & humanoid delays
4	result_filter_agent.py	Strict filter: keep results mentioning both company AND designation
5	llm_processor.py	Progressive extraction: send top 3 results, then next 5 if needed
6	verification_agent.py	Verify LLM output: company_match + designation_match + current_role

Step	Module	Action
7	confidence_score.py	Score result 0–100 based on match flags + source domain tier
8	routes/search.py	Return best result or trigger relaxed pass if confidence < 60

3. Setup & Installation

3.1 Prerequisites

Requirement	Version	Notes
Docker & Docker Compose	v24+	Required for running SearXNG and the API
Python	3.11+	Only needed for local development (no Docker)
Groq API Key	Free tier	Register at console.groq.com
Git	Any	To clone the repository

3.2 Environment Variables

Create or edit the file app/.env with the following variables:

```
# Required — Groq API Key
GROQ_API_KEY=gsk_your_key_here

# SearXNG instance URL (auto-configured via Docker Compose)
SEARXNG_BASE_URL=http://localhost:8888

# Max concurrent rows during bulk search (default: 5)
BULK_CONCURRENCY=5

# Set to true to test without real API keys
MOCK_MODE=false

# Allowed CORS origins (comma-separated)
FRONTEND_ORIGINS=http://localhost:3000,http://localhost:8000
```

3.3 Docker Deployment (Recommended)

This is the simplest and recommended way to run the full stack.

```
# 1. Clone the repository
git clone <your-repo-url>
cd "Person Name Finder searXNG"

# 2. Set your Groq API key
echo GROQ_API_KEY=gsk_your_key > .env

# 3. Start both services
docker-compose up --build

# The API will be available at:
#   http://localhost:8010      (API root)
```

```
#   http://localhost:8010/ui   (Web UI)
#   http://localhost:8010/docs (Swagger docs)
#
# SearXNG will be available at:
#   http://localhost:8888
```

3.4 Local Development (No Docker)

```
# 1. Install dependencies
cd app
pip install -r requirements.txt

# 2. Start SearXNG separately (Docker still needed for SearXNG)
docker run -p 8888:8080 searxng/searxng

# 3. Run the FastAPI server
python main.py

# OR using uvicorn directly:
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# Windows: use run_server.bat
```

Note

The Dockerfile uses gunicorn with 4 UvicornWorker processes for production. For development, the --reload flag enables hot-reloading on code changes.

4. API Reference

The API is built with FastAPI and exposes interactive Swagger documentation at /docs and ReDoc at /redoc when running.

4.1 GET /

Health check endpoint — returns API status and available endpoints.

```
GET http://localhost:8010/

Response 200:
{
  "message": "Person Name Finder API is running.",
  "ui_url": "/ui/index.html",
  "endpoints": {
    "search": "/search (POST)",
    "bulk_search": "/bulk-search (POST)"
  }
}
```

4.2 POST /search

Performs a single company + designation search and returns the matched person's details.

Request Body

Field	Type	Description
company	string	Company name to search (e.g., "Google", "PureLogics")
designation	string	Job title or role (e.g., "CEO", "Chief Technology Officer")

Response Body

Field	Type	Description
company	string	Echo of the requested company name
designation	string	Echo of the requested designation
name	string	Full name of the matched person, or "Not Found"
source	string	URL of the web page the name was found on
confidence	integer	Score from 0–100 indicating result certainty
status	string	Human-readable result status (e.g., "Found", "Not Found")

Example

```
POST /search
Content-Type: application/json

{
  "company": "OpenAI",
  "designation": "CEO"
}

Response 200:
{
  "company": "OpenAI",
  "designation": "CEO",
  "name": "Sam Altman",
  "source": "https://www.linkedin.com/in/sama",
  "confidence": 95,
  "status": "Found"
}
```

4.3 POST /bulk-search

Accepts a CSV file containing multiple company/designation pairs and returns a downloadable CSV with results.

Request

Content-Type: multipart/form-data with a file field named "file".

CSV Input Format

```
company,designation
Google,CEO
Microsoft,CTO
Apple,Chief Executive Officer
Tesla,Founder
```

CSV Output Format

```
company,designation,name,source,confidence,status
Google,CEO,Sundar Pichai,https://about.google/intl/en/,95,Found
Microsoft,CTO,Kevin Scott,https://linkedin.com/in/...,88,Found
```

Concurrency

Bulk rows are processed concurrently. The BULK_CONCURRENCY environment variable (default: 5) controls how many rows are processed simultaneously to avoid overwhelming the search engine or hitting API rate limits.

5. Core Modules

5.1 Query Generator — query_generator.py

Generates 10 different search query variants for a given company and designation to maximise the chance of finding relevant results across different search engines and page types.

Generated Query Templates

#	Query Template
1	"<company>" <designation> — Quoted company name for exact match
2	<designation> at <company> — Natural language phrasing
3	<company> <designation> name — Direct name-finding intent
4	"<company>" <designation> LinkedIn 2014 LinkedIn-targeted
5	<company> who is the <designation> — Question form
6	<designation> "<company>" site:linkedin.com — LinkedIn site-specific
7	<company> <designation> profile — Profile page targeting
8	"<company>" current <designation> 2014 Emphasises currency
9	<company> <designation> contact — Contact page targeting
10	<designation> <company> — Reversed order

5.2 SearXNG Client — searxng_client.py

An async HTTP client that queries the self-hosted SearXNG instance with humanoid behaviour to avoid bot detection.

Key Behaviours

- Random delays: 0.5–2.0 seconds on first attempt, 2–5 seconds on retries
- Random User-Agent rotation across 5 different browser fingerprints
- Random engine selection: 2–3 engines chosen from a pool of 7 (DuckDuckGo, Brave, Qwant, Bing, Google, Startpage, Yahoo)
- Up to 3 retries per query before returning an empty result set
- Timeout of 25 seconds per HTTP request

5.3 LLM Processor — llm_processor.py

Calls the Groq API to extract a person's name from a search result snippet. Uses a structured JSON prompt and handles model rotation on rate-limit errors.

Model Rotation Pool

Model	RPM Limit	RPD Limit	Notes
llama-3.1-8b-instant	30	14,400	Primary — highest daily limit
qwen/qwen3-32b	60	1,000	Fallback #1
moonshotai/kimi-k2-instruct	60	1,000	Fallback #2
llama-4-scout-17b-16e	30	1,000	Fallback #3
llama-3.3-70b-versatile	30	1,000	Fallback #4 — highest quality

Progressive Extraction Strategy

To conserve LLM API quota, extraction is done progressively rather than sending all results at once:

- Step 1 — Quick scan: send the top 3 filtered results to the LLM one by one. If any returns a valid name, return immediately.
- Step 2 — Deep scan: if Step 1 finds nothing, send the next 5 results. Return the first valid name found.
- Step 3 — Fallback: if still nothing, the caller triggers the relaxed search pass.

LLM JSON Output Format

```
{
  "name": "Full Name or 'Unknown'",
  "company_match": true | false,
  "designation_match": true | false,
  "current_role": true | false,
  "reasoning": "One sentence explanation"
}
```

5.4 Result Filter Agent — result_filter_agent.py

Filters raw search results before sending them to the LLM, removing irrelevant or outdated entries. Supports two filtering modes:

Strict Filter (Pass 1)

- Result must mention the company name (with CamelCase splitting and suffix removal)
- Result must mention the designation or a recognised alias (e.g., CEO = Chief Executive Officer)
- Results containing exclusion keywords (former, ex-, previously, retired) are removed

Relaxed Filter (Pass 2 Fallback)

- Only requires the company name to appear — designation is not required
- Still excludes former/ex- results

- Used when the strict filter returns zero results

Supported Designation Aliases

The filter recognises common abbreviation-to-full-name mappings, including: CEO ↔ Chief Executive Officer, CTO ↔ Chief Technology Officer, CFO ↔ Chief Financial Officer, COO ↔ Chief Operating Officer, MD ↔ Managing Director, VP ↔ Vice President, Founder ↔ Co-Founder, and more.

5.5 Verification Agent — verification_agent.py

Validates the LLM's parsed output before accepting it as a result. Two modes mirror the two search passes:

Strict Verification

- name must be a non-empty string (not 'Unknown', 'N/A', or 'None')
- company_match must be true
- designation_match must be true
- current_role must be true

Relaxed Verification

- name must be a non-empty, valid string
- company_match must be true
- At least one of designation_match or current_role must be true

5.6 Confidence Scoring — confidence_score.py

Computes an integer score from 0–100 for each candidate result based on four weighted factors:

Factor	Points	Condition
Company Match	+40	LLM confirmed the result is about the correct company
Designation Match	+30	LLM confirmed the designation matches (any alias form)
Snippet Quality	+10	Non-empty snippet with more than 20 characters
Source Trust Bonus	+5 to +20	Based on domain tier (see below)
Not Current Role Penalty	−30	Applied when LLM flags the role as former/past

Source Trust Tiers

Tier	Domain Examples	Category	Bonus
1	linkedin.com, company's own site	Professional profile / official site	+20
2a	crunchbase.com, bloomberg.com, forbes.com	Business databases	+15
2b	zoominfo.com, clutch.co, g2.com	B2B directories	+13
3	reuters.com, glassdoor.com, indeed.com	News wires / job sites	+10
4	techcrunch.com, bbc.com, cnbc.com	General tech/news media	+8
—	Unknown / other domains	Unrecognised source	+5

6. Search Strategy

The search follows a two-pass strategy designed to maximise recall while minimising unnecessary LLM API calls.

6.1 Pass 1 — Strict Search

- Up to 3 query variants are tried sequentially
- Each query fetches results from SearXNG (top 10 results)
- Results are strictly filtered (company AND designation must appear)
- Progressive LLM extraction: top 3 results first, then next 5
- If confidence ≥ 90 at any point, return immediately (early exit)
- If best confidence ≥ 60 after 3 queries, return the best result

6.2 Pass 2 — Relaxed Search (Fallback)

- All 10 query variants are tried
- Results are filtered using relaxed mode (only company required)
- Progressive LLM extraction with relaxed verification
- A 10-point penalty is applied to confidence scores in this pass
- Returns if confidence ≥ 75

6.3 Known Corrections

A hardcoded dictionary of company → designation → name mappings is checked before any search. Matching entries return instantly with confidence 100 and status "Verified correction". This is useful for companies where search results are unreliable or outdated.

6.4 Mock Mode

When MOCK_MODE=true, all search and LLM calls are bypassed. A random name from a built-in list of well-known executives is returned after a simulated 0.8-second delay. Useful for front-end testing and demos without consuming API quota.

7. Frontend

The frontend is a self-contained single-page application (SPA) consisting of three files: index.html, app.js, and style.css. It is served directly by the FastAPI backend at the /ui route.

7.1 Pages and Sections

Single Search

- Two text inputs: Company and Designation
- Search button triggers POST /search
- Result card shows: name, confidence badge, source URL, status
- Results are also appended to the persistent results table at the bottom

Bulk Search

- Drag-and-drop or click-to-upload CSV file input
- Displays the selected file name and validates the .csv extension
- Bulk Search button triggers POST /bulk-search (multipart upload)
- On completion, a download button appears for the result CSV
- Results are also shown in the results table inline

Results Table

- Persistent table accumulates all search results during the session
- Columns: Company, Designation, Name, Confidence, Source, Status
- Download All button exports the full table as a CSV

7.2 API Communication

The frontend uses the Fetch API to communicate with the backend. The base URL is inferred from window.location.origin, so it works regardless of whether the app is accessed via localhost or a remote server. CORS is configured on the backend to allow the frontend's origin.

8. Configuration Reference

8.1 Environment Variables

Variable	Default	Required	Description
GROQ_API_KEY	—	Yes	Groq API key from console.groq.com
SEARXNG_BASE_URL	http://localhost:8888	No	URL of the SearXNG instance
BULK_CONCURRENCY	5	No	Max concurrent bulk search rows
MOCK_MODE	false	No	Return fake data without API calls
FRONTEND_ORIGINS	localhost:3000,...	No	Allowed CORS origins (comma-separated)

8.2 Docker Compose Ports

Service	Port Mapping	Description
api	8010 → 8000	FastAPI application (host:container)
searxng	8888 → 8080	SearXNG search engine (host:container)

8.3 Rate Limiting

SlowAPI is configured on the FastAPI app using the client IP address as the rate limit key. The default rate limit is applied per-route. To customise limits, modify the @limiter.limit() decorator on each route in search.py and bulk_search.py.

9. Dependencies

9.1 Python Packages

Package	Version	Purpose
fastapi	0.110.0	Web framework for building the REST API
uvicorn[standard]	0.29.0	ASGI server for running FastAPI
httpx	0.27.0	Async HTTP client for SearXNG requests
python-dotenv	1.0.1	Load environment variables from .env file
groq	0.5.0	Official Groq Python SDK
python-multipart	0.0.9	Multipart form handling for file uploads
pandas	2.2.1	CSV parsing and serialization for bulk search
aiofiles	23.2.1	Async file I/O utilities
duckduckgo-search	5.3.1b1	DuckDuckGo client (fallback search)
slowapi	0.1.9	Rate limiting middleware for FastAPI

9.2 Docker Images

Image	Purpose
python:3.11-slim	Base image for the FastAPI API container
searxng/searxng	Official SearXNG self-hosted search engine

10. Troubleshooting

10.1 Common Issues

Problem	Solution
"Not Found" for all queries	Check GROQ_API_KEY is set and valid. Enable MOCK_MODE=true to verify UI is working.
SearXNG returns no results	Verify SearXNG is running at SEARXNG_BASE_URL. Check docker-compose logs for the searxng service.
Rate limit errors in logs	The system auto-rotates models. If persistent, reduce BULK_CONCURRENCY or add a delay between requests.
Low confidence scores	The company or designation may be uncommon. Try adding it to the COMPANY_CORRECTIONS dict in search.py.
CSV upload fails	Ensure the CSV has exactly the columns: company, designation (case-insensitive). Remove BOM if present.
CORS errors in browser	Add your frontend URL to FRONTEND_ORIGINS in .env and restart the server.
Docker port conflict	Change the host port in docker-compose.yml (e.g., "8011:8000" instead of "8010:8000").

10.2 Log Files

The application writes structured logs to stdout (captured by Docker). Two log files are also written during development:

- app/server.log — Full server request/response log
- app/crash.log — Exception traces and startup errors

To view Docker logs in real time:

```
# All services
docker-compose logs -f

# API only
docker-compose logs -f api

# SearXNG only
docker-compose logs -f searxng
```

11. Security Considerations

> **Important**
> The .env file contains your Groq API key. Never commit this file to version control. Add .env to your .gitignore.

11.1 Recommendations

- Rotate your Groq API key periodically from console.groq.com
- Restrict FRONTEND_ORIGINS to your specific domain in production (do not use wildcard *)
- Run the Docker containers behind a reverse proxy (nginx / Caddy) with HTTPS in production
- Set BULK_CONCURRENCY to a low value (3–5) to avoid inadvertently triggering rate limits
- Do not expose the SearXNG port (8888) publicly — it is only needed internally by the API
- Consider adding authentication middleware to the /bulk-search endpoint for production deployments

12. Extending the Application

12.1 Adding Known Corrections

To hardcode a result for a specific company, edit the COMPANY_CORRECTIONS dictionary in app/routes/search.py:

```
COMPANY_CORRECTIONS = {
    "YourCompany": {
        "CEO": "Jane Doe",
        "Chief Executive Officer": "Jane Doe",
        "CTO": "John Smith",
    },
}
```

12.2 Adding a New Groq Model

Add the model ID to the _MODELS list in app/services/llm_processor.py. Models are tried in order, starting at a round-robin index:

```
_MODELS = [
    "llama-3.1-8b-instant",   # Primary
    "your-new-model-id",       # Add here
    "llama-3.3-70b-versatile", # Last resort
]
```

12.3 Adding Designation Aliases

Extend the _DESIGNATION_ALIASES list in app/agents/result_filter_agent.py to support new role abbreviations:

```
_DESIGNATION_ALIASES = [
    ["ceo", "chief executive officer", "chief exec"],
    ["your-abbr", "your full title"],  # Add new alias group
    ...
]
```

12.4 Adding Source Trust Domains

Add domains to the _DOMAIN_TRUST list in app/utils/confidence_score.py to assign appropriate trust scores:

```
_DOMAIN_TRUST = [
    (["linkedin.com"], 20),
    (["your-trusted-domain.com"], 15),  # Add here
    ...
]
```

Appendix: Quick Reference

API Endpoints Summary

Method	Endpoint	Input	Output
GET	/	—	API status and endpoint list
GET	/ui	—	Serves the web UI (index.html)
GET	/docs	—	Interactive Swagger API documentation
POST	/search	JSON {company, designation}	JSON result with name + confidence
POST	/bulk-search	CSV file upload	Downloadable CSV with all results

Confidence Score Thresholds

Range	Status Label	Meaning	Pass
90–100	Found	Very high confidence — early exit triggered	1
70–89	Found	High confidence result	1 or 2
60–69	Found / Possible	Moderate confidence — returned from strict pass	1
40–59	Possible match	Lower confidence — returned from relaxed pass	2
0–39	Not Found	No reliable match found	—